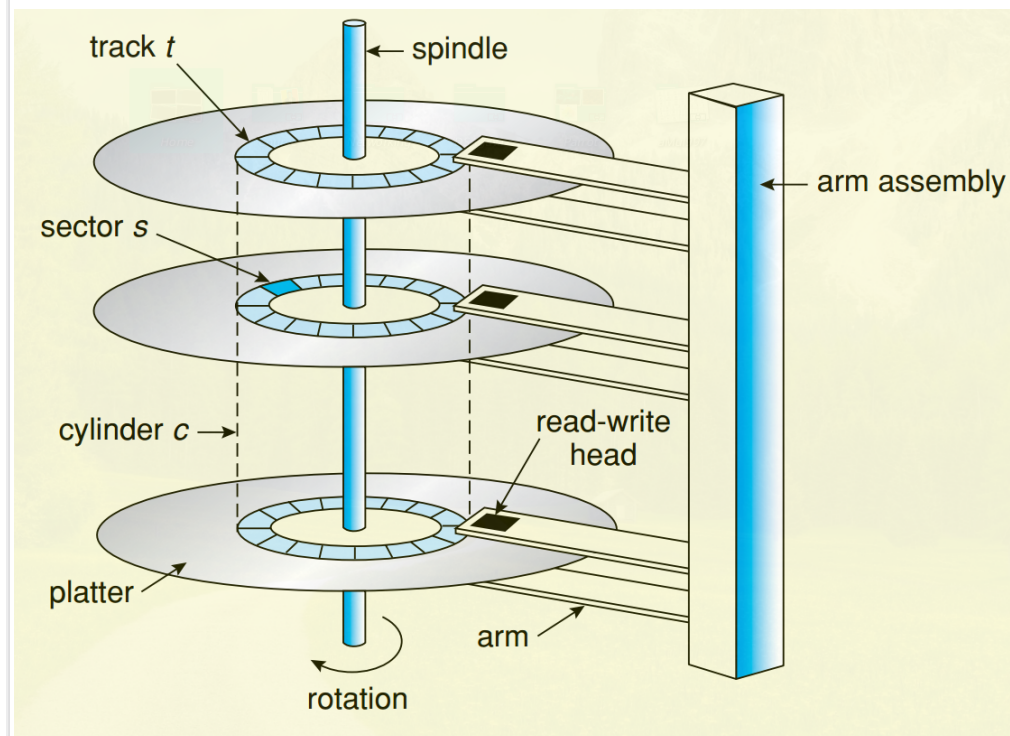


Chap10_Mass Storage Systems

Chap10_Mass Storage Systems

Objectives:

- Physical structure
- Performance
- Evaluating I/O scheduling algorithms
- OS services for mass storage.



Magnetic Disks Mass Storage Structures (Different from magnetic tapes, these are typical HDD we use)

- HDDs spin platters of magnetically-coated material under moving read-write heads
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time (random-access time)** consists of two parts
 - **Seek time** is time to move disk arm to desired cylinder
 - Time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface. That's bad

Head crash results from disk head making contact with the disk surface -- that's bad

- **Access Latency** = **Average access time** = average seek time + average latency
- Average I/O time = average access time + (amount to transfer / transfer rate) + controller overhead
- **Controller: two parts:**
 - **Host controller**, at the computer end of the bus
 - **Disk controller**, built in to each disk drive
 - It also has a **built in cache**, data transfers as the disk drive happens between the cache and the disk surface
- To perform a disk I/O operation,
 - The computer places a command into the host controller, typically using memory-mapped I/O ports.
 - The host controller then sends the command via messages to the disk controller.
 - The disk controller operates the disk-drive hardware to carry out the command.
- A disk drive is attached to a computer by a set of wires called an I/O bus.
- Several kinds of buses are available, including **advanced technology attachment (ATA)**, **serial ATA (SATA)**, **eSATA**, **universal serial bus (USB)**, and **fibre channel (FC)**

SSDs

- NVM
- Many variations -
 - **DRAM with a battery** to allow it to maintain its state in a power failure through flash-memory technologies like single-level cell (SLC) and multilevel cell (MLC) chips
- Because SSDs can be much faster than magnetic disk drives, standard bus interfaces can cause a major limit on throughput.
 - Some SSDs are designed to connect directly to the system bus (PCI, for example).
- **Adv.**
 - No moving parts
 - Less power consumption
 - No seek time or latency
- **Disad**
 - Less capacity
 - Shorter life span
 - Expensive

NVM (Non Volatile Memory) Devices

- Maybe have shorter life span – need careful management
- Less capacity
- But much faster
- Busses can be too slow → connect directly to PCI for example
- No moving parts, so no seek time or rotational latency
- Characteristics:
 - Read and written in "**page**" **increments** but can't overwrite in place.
 - Must be erased first, which happens in larger blocks

- Must be erased first, which happens in larger blocks
- Limited number of times, can it be erased before wornout. ~100,000
- Life span measured in **DRIVE WRITES PER DAY(DWPD)**
 - A 1TB NAND drive with rating of 5DWPD is expected to have 5TB per day written within warrantee period without failing
- No overwrits pages end up with mix of valid and invalid data
- **FLASH TRANSLATION LAYER table** to track which logical blocks are valid
- **GARBAGE COLLECTION** to free invalid page source



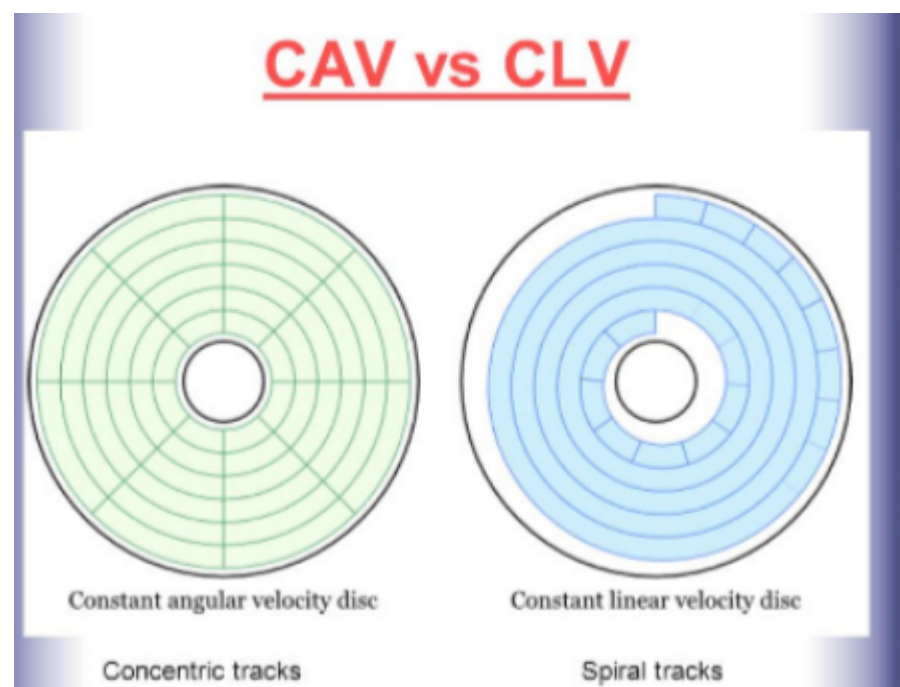
NAND block with valid and invalid pages

	Transfer Rate	Seek Time	Latency	Range		
HDD	6 Gb/sec (Effective 1Gb/sec)	3ms to 12 ms	• Based on spindle speed = 1/(RPM/60) = 60/RPM • Average latency = (1/2)*latency	30GB to 3TB [per drive]		

Chap10_Disk_Structures_&_Disk_Scheduling

- **Disk structures**

- 1-dimensional arrays of logical blocks
- **Logical block** is the smallest unit of transfer, typically 512 bytes,
 - Low level formatted(1024 or more bytes LB) means LOWER NUMBER OF BLOCKS, which means BIGGER BLOCK SIZE.
- The one-dimensional array of logical blocks is mapped onto the sectors of the disk sequentially.
 - Sector 0 is the first sector of the first track on the outermost cylinder.
 - The mapping proceeds in order through that track, then through the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.
- In practice, it is difficult to perform the logical block into an old style disk address, for two reasons
 1. Defective sectors (solution, mapping hides this by substituting a spare sector from elsewhere)
 2. Number of sectors per track is not a constant
- Second problem close-up
 - **CLV (Constant Linear Velocity)**
 - Density of BITS per TRACK is constant ()
 - As we move from outer zones to inner zones, the number of sectors per track decreases.
 - The drive increases its rotation speed as the head moves from the outer to the inner tracks to keep the same rate of data moving under the head.
 - This method is used in CD-ROM and DVD-ROM drives.
 - **CAV(Constant Angular Velocity)**
 - Density of bits decreases from inner tracks to outer tracks to keep data rate constant
 - Used in hard disks.
 - [Slide on disks](#)



- **DISK attachment**

- **Host Attached Storage**

- Local I/O ports using I/O bus architecture called IDE or ATA.
 - Can also use more sophisticated I/O architectures such as **FC (Fibre Channel)**.
 - Also **FC-AL (FC Arbitrated Loop)**

- **Network Attached Storage**

- Accessed remotely over a data network via a **Remote Procedure Call (RPC)** such as NFS for UNIX or CIFS for Windows
 - **RPCs are carried via TCP or UDP over an IP network. (OneDrive could be an example)**
 - Usually implemented as a **RAID array** with software that implements the RPC interface.

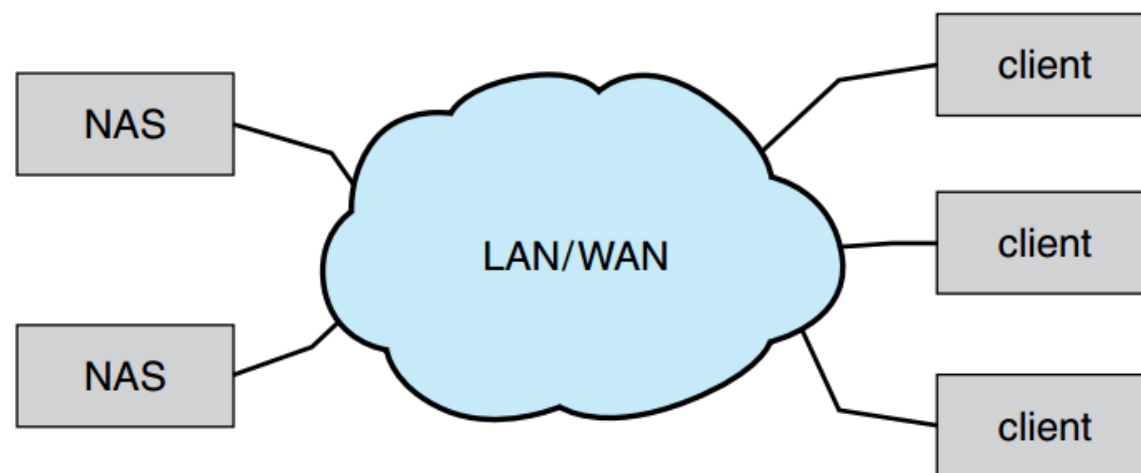
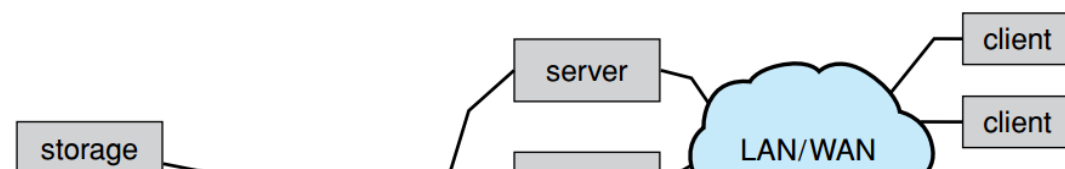


Figure 10.2 Network-attached storage.

- - Less efficient than direct attached storage options.
 - Also it consumes bandwidth of data network, so **SAN below** are a better option

- **Storage Area Network**

- A private network which **uses storage protocols rather than networking protocols**
 - More flexible, storage can be **dynamically allocated to hosts**.
 - **SAN switch** allows or prohibits access between the hosts and the storage.
 - FC commonly used, although **iSCSI** use is increasing, **InfiniBand** for high speed interconnection networks for servers and storage units.



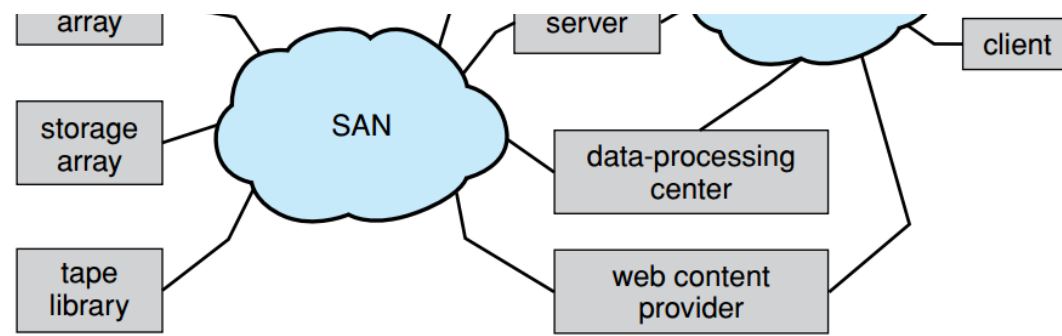


Figure 10.3 Storage-area network.

• Disk Scheduling Algorithms

- The **disk bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.
- Whenever a process needs I/O to or from the disk, it issues a system call to the operating system.
- The request specifies several pieces of information:
 - • Whether this operation is input or output
 - • What the disk address for the transfer is
 - • What the memory address for the transfer is
 - • What the number of sectors to be transferred is
- **Total head movement** is the number of cylinder traversals the head had to make to get a particular sector
- **Algorithms**

▪ FCFS

- Problem Snot always the fastest
- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

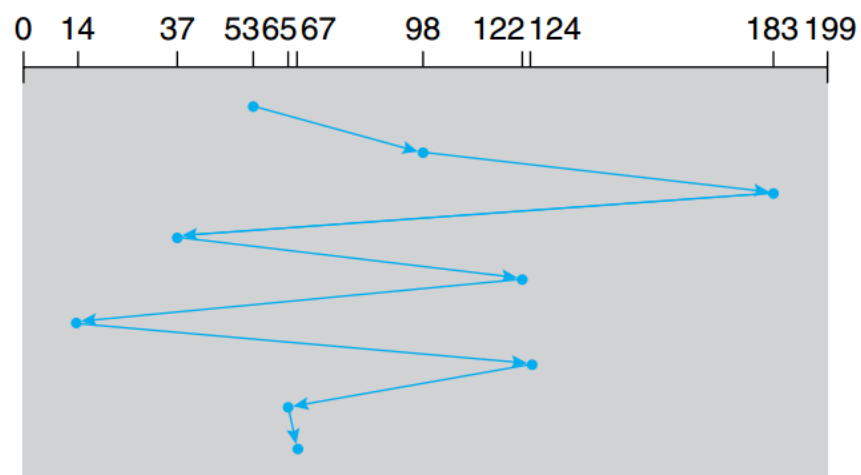


Figure 10.4 FCFS disk scheduling.

-
- SSTF(Shortest Seek Time first)

- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

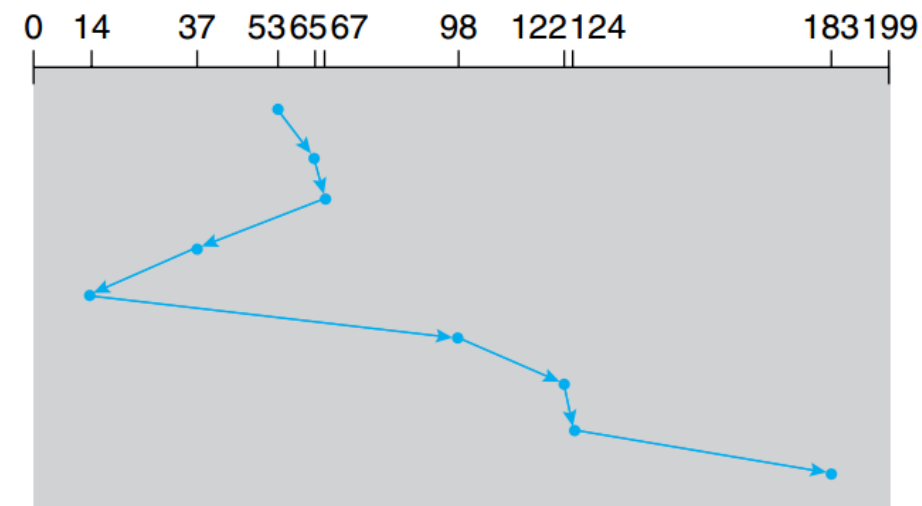


Figure 10.5 SSTF disk scheduling.

- Problem is **starvation**
- **SCAN** (Scanning the whole disk before changing direction and scanning backwards)
 - Seek time could be double for a I/O request popping up behind the head.
 - Also called the **elevator algorithm**.
- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

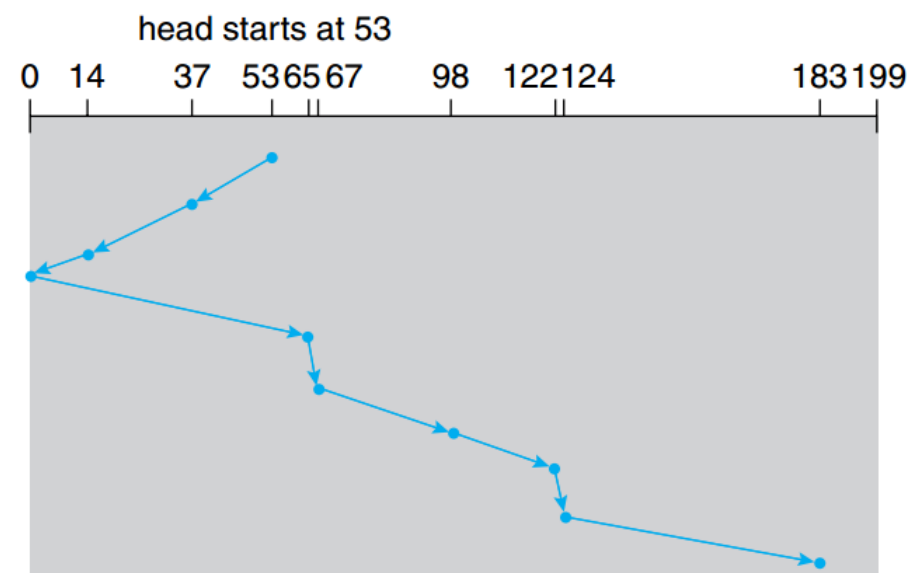


Figure 10.6 SCAN disk scheduling.

- Circular-SCAN (SCAN variant with no backward scan but again from starting)

- To avoid the problem of the SCAN's algorithm, the CSCAN reverts the head back to the starting position.
- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

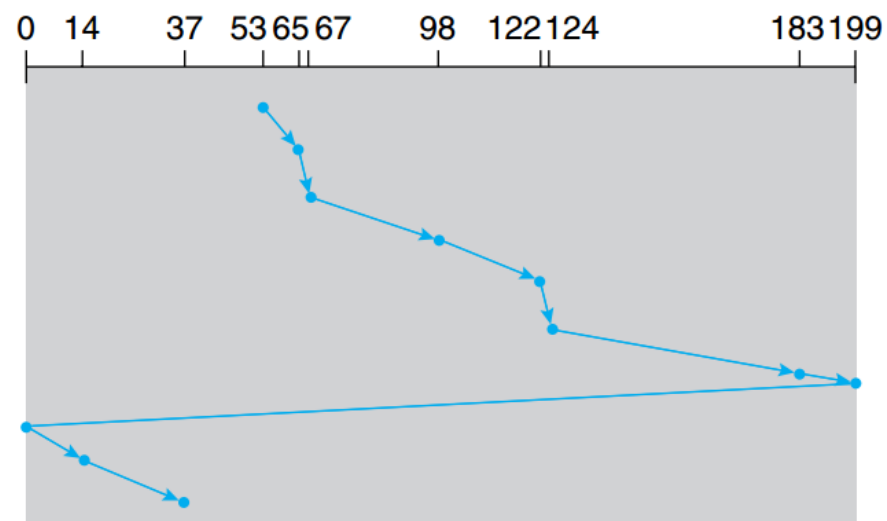


Figure 10.7 C-SCAN disk scheduling.

- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53

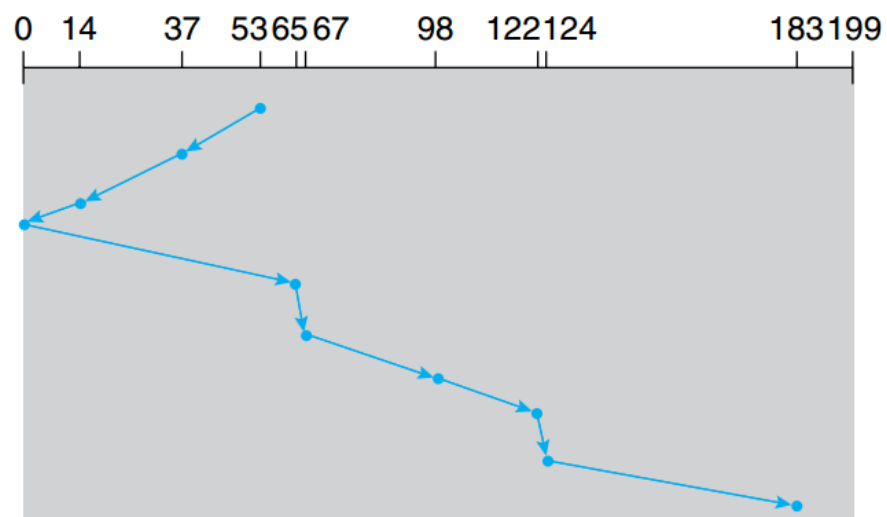
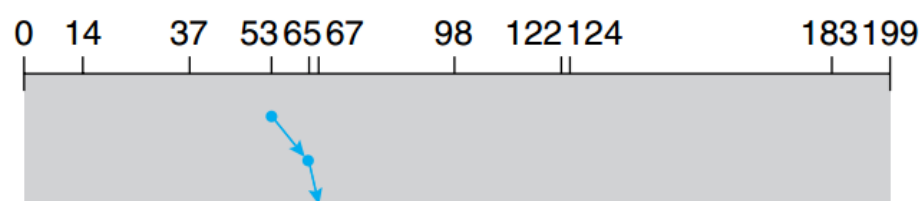


Figure 10.6 SCAN disk scheduling.

- LOOK (SCAN with the head moving only till the last address needed, not necessarily to the end of disk)
- C-LOOK (LOOK with SCAN C-SCAN analogy)

- queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



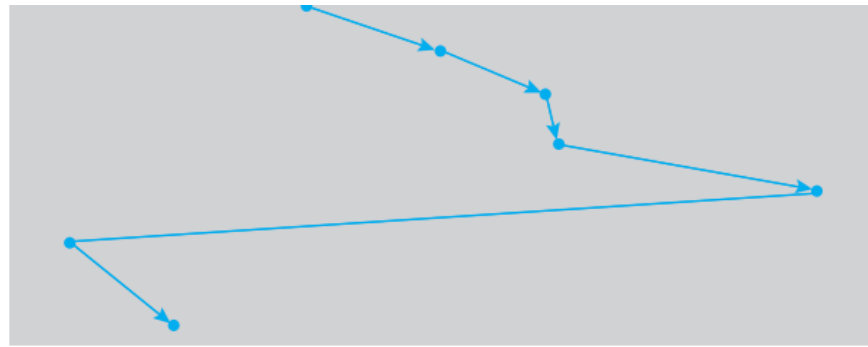


Figure 10.8 C-LOOK disk scheduling.

DISK SCHEDULING and SSDs

The disk-scheduling algorithms discussed in this section focus primarily on minimizing the amount of disk head movement in magnetic disk drives. SSDs—which do not contain moving disk heads—commonly use a simple FCFS policy. For example, the Linux **Noop** scheduler uses an FCFS policy but modifies it to merge adjacent requests. The observed behavior of SSDs indicates that the time required to service reads is uniform but that, because of the properties of flash memory, write service time is not uniform. Some SSD schedulers have exploited this property and merge only adjacent write requests, servicing all read requests in FCFS order.

• Selection among Scheduling algorithms

- Requests for disk service can be greatly influenced by the file-allocation method.
 - A program reading a **contiguously allocated file** will **generate several requests that are close together on the disk**, resulting in **limited head movement**.
 - A **linked or indexed file**, in contrast, may include blocks that are widely scattered on the disk, resulting in **greater head movement**.
 - The location of directories and index blocks is also important.
 - Since every file must be opened to be used, and **opening a file requires searching the directory structure**, the **directories will be accessed frequently**.
 - Suppose that a **directory entry is on the first cylinder** and a file's data are on the **final cylinder**. In this case, the **disk head has to move the entire width of the disk**.
- The scheduling algorithms described here consider **only seek distances, not rotational latency**.
- Difficult for OS to schedule for improved rotational latency.
- I/O is not the only issue, if it were the OS would gladly turn the responsibility of disk scheduling to the disk hardware, but OS have other constraints too such as:
 - **Demand Paging** may take priority over application I/O and writes are more urgent than reads if the cache is running out of free pages.

- Demand paging may take priority over application I/O, and writes are more urgent than reads if the cache is running out of free pages.
- To accommodate such requirements, an operating system may choose to do its own disk scheduling and to spoon-feed the requests to the disk controller, one by one, for some types of I/O.

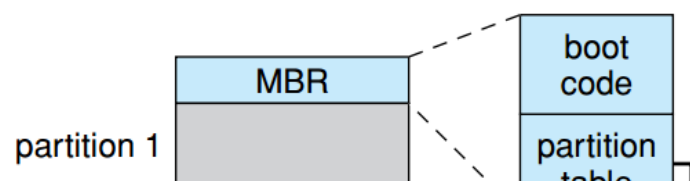
Chap10_Disk_Mngt

Disk Management

- **Disk formatting**
 - Dividing into sectors that the disk controller can read and write, called **low level formatting** or **physical formatting**. Done at factory.
 - HEADER---DATA---TRAILER
 - The above is the data structure for a sector.
 - The Header and trailer have info such as sector number and **ECC (Error Correcting Code)** just like CRC in networks
 - Sector size may be 512, 256 or 1024 etc bytes. Larger sector size means less sectors and less data used up by header and trailer, so more data available for use.
 - Before the disk can hold data, the OS still needs to record its own data structures on the disk, which is done in two steps:
 1. **Partitioning the disk**, so the OS can treat each partition as though it were a separate disk
 2. **Logical Formatting**, the creation of file systems.
 - To increase efficiency, most file systems group blocks together into larger chunks, frequently called clusters.
 - Disk I/O is done via blocks, but file system I/O is done via clusters, effectively assuring that I/O has more sequential-access and fewer random-access characteristics ??????????????????
 - **Raw disk** can be used to give special programs the ability to use the partition as a large sequential array of logical blocks, **without any file system data structures**.
 - Enables user to control the **exact disk location where each database is stored**
 - Raw I/O **bypasses all the file-system services**, such as the buffer cache, file locking, prefetching, space allocation, file names, and directories.

Boot Block

- **Bootstrap program** initializes all aspects of the system.
 - Finds the kernel on disk
 - Loads kernel into memory
 - Jumps to initial address to begin the OS execution
 - Stored in **Read-Only Memory(ROM)**
 - Because, no initialization required, it is at a fixed location that the processor can start executing during bootup
 - no threat of infection
 - Usually, **not all Bootstrap is stored in ROM, a tiny bootstrap loader program is stored in ROM, whose job is to bring the full bootstrap program into disk**
 - This program can be changed
 -



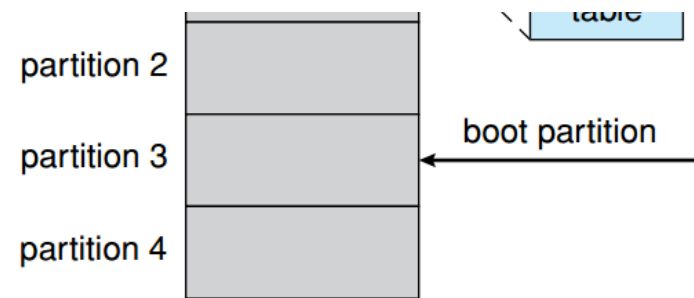


Figure 10.9 Booting from disk in Windows.

• Booting process in Windows

- The **boot partition** is identified, the partition which contains device drivers and OS
 - The **first sector of this disk** contains the **MBR (Master Boot Record)**
 - The MBR contains the **table listing the partitions for the harddisk and a flag indicating which partition the system is to be booted from.**
- Executing the **ROM bootstrap code**, which reads **the full bootstrap code, stored in the first sector (boot sector) of the partition, not the disk**
- Loading subsystems and modules

Bad Blocks

- Defective sectors,
- Can be handled manually or automatically
- Bad Blocks are flagged as **unusable**
- The controller maintains a **list of bad blocks on the disk**
 - **The list is initialized during low level formatting**
 - **Updated throughout its lifetime**
- A typical bad-sector transaction might be as follows:
 - The operating system tries to read logical block 87.
 - The controller calculates the ECC and finds that the sector is bad. It reports this finding to the operating system.
 - The next time the system is rebooted, a special command is run to tell the controller to replace the bad sector with a spare.
 - After that, whenever the system requests logical block 87, the request is translated into the replacement sector's address by the controller.
- Two methods typically used to solve bad blocks
 1. **Sector sparing or Forwarding**
 - Rdirection from Bad Blocks could **invalidate any optimization by OS disk scheduling algorithm,**

- so a few sectors are provided on each cylinder or a spare cylinder for this purpose

2. Sector slipping

- Suppose sector 17 is defected, and sector 202 is the first available spare sector,
- So all sectors from 17 to 201 are slipped one sector down, 17-18, 18-19.....201-202
- Replacement is not always automatic, **Soft errors may trigger it, a hard error might be irrecoverable**

Chap10_Swap

- In practice, very few modern operating systems implement swapping in this fashion.
- Rather, systems now combine swapping with virtual memory techniques (Chapter 9) and swap pages, not necessarily entire processes
- Swap-space management is another low-level task of the operating system.
- The main goal for the **design and implementation of swap space is to provide the best throughput for the virtual memory system.**
- **Swap Space Use**
 - Swap Space is used in various ways by different operating systems, depending on the memory-management algorithms in use.
 - Can hold entire process image
 - Or Paging systems may simply store pages that have been pushed out of memory
 - Safer to overestimate than to underestimate the amount of swap space required, because if a system runs out of swap space it may be forced to abort processes, Over estimation only wastes space but no harm otherwise
 - Examples
 - Solaris- Swap space = Virtual memory-pageable physical memory
 - Linux- Double the physical memory, although flexible
- **Swap Space location**
 - Two places:
 - **Carved out of normal file system**
 - Should follow normal file system routine
 - . Navigating the directory structure and the disk allocation data **structures takes time and (possibly) extra disk accesses.**
 - **External fragmentation** can **greatly increase swapping times** by forcing multiple seeks during reading or writing of a process image.
 - **Separate raw disk partition**
 - **No file system** or directory structure placed, just a **swap-space manager.**
 - Uses algorithms **optimized for speed rather than storage** efficiency.
 - **Internal fragmentation** may increase, but this trade-off is acceptable because the life of data in the swap space generally is much shorter than that of files in the file system.
 - Linux can uses both types

• Swap Space Management

- Swap space -
 - 4kb page slots
 - There can be multiple swap areas
 - Each swap area has a **swap map**
 - The swap map is an integer array with values indicating the number of mappings to the swapped pages.

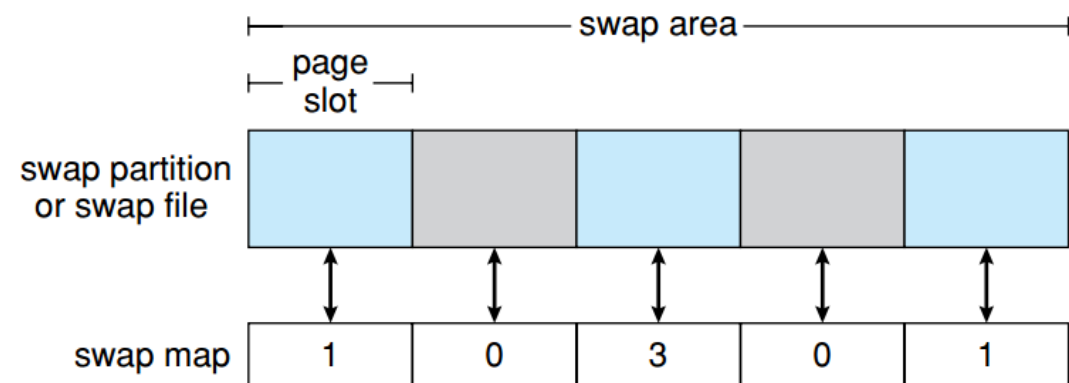


Figure 10.10 The data structures for swapping on Linux systems.

-
- Swap is used as a backing store only for pages of **anonymous memory, which has stacks, heap, uninitialized data**. Otherwise, if a file exists on the filesystem, it is efficient to reread the page from there than writing it to swap space.
- Solaris - only allocates swap when a page is paged out, rather than when virtual memory is created.
- Linux is similar to Solaris in that swap space is used only for anonymous memory—that is, memory not backed by any file.

Chap10_RAID

RAID Structure

- Used to address performance and reliability issues

-

STRUCTURING RAID

RAID storage can be structured in a variety of ways. For example, a system can have disks directly attached to its buses. In this case, the operating system or system software can implement RAID functionality. Alternatively, an intelligent host controller can control multiple attached disks and can implement RAID on those disks in hardware. Finally, a **storage array**, or **RAID array**, can be used. A RAID array is a standalone unit with its own controller, cache (usually), and disks. It is attached to the host via one or more standard controllers (for example, FC). This common setup allows an operating system

or software without RAID functionality to have RAID-protected disks. It is even used on systems that do have RAID software layers because of its simplicity and flexibility.

Reliability via Redundancy

- **Mirroring** : duplicating every disk
 - **Mean time to failure of a mirrored volume** depends on two factors
 - **Mean time to failure of an individual disks.**
 - **Mean time to repair**, time taken to replace a failed disk and to restore data on it
 - if the mean time to failure of a single disk is 100,000 hours and the mean time to repair is 10 hours, the mean time to data loss of a mirrored disk system is $100,000 \text{sq}/(2 * 10) = 500 * 10^6$ hours, or 57,000 years!
- we cannot really assume that disk failures will be independent.
 - **Power failures and natural disasters** can result in damage to both disk
 - **Solution**
 - One is to write one copy first, then the next.
 - Another is to add a solid state NVRAM (Non-Volatile RAM) cache to the RAID array, assuming NVRAM has some kind of error protection and correction such as ECC or mirroring
 - Manufacturing defects can also cause correlated failures

Performance via Parallelism

- Read requests handling **doubles** after mirroring, although, the transfer rate of each read is the same as in a single-disk system, but the number of reads per unit time has doubled.
- **DATA Striping**
 - **BIT level striping**, storing each bit of a byte on a different disk, that can be accessed parallelly, increasing read eight time. The number of disk should be a multiple of 8 or divide 8
 - **BLOCK level striping(most common)**, where instead of bits, blocks of a file are stored in different disks $(i \bmod n)+1$
 - **BYTE level striping**
 - **Sector level striping**
- Increases throughput of multiple small accesses by **load balancing**
- Reduces response time of large accesses

RAID Levels

- Striping provides No improvement in reliability, only data transfer rates, while Mirroring provides improvement in reliability, not data transfer rates, and it is expensive too
- shows them pictorially (in the figure, **P indicates error-correcting bits and C indicates a second copy of the data**). In all cases depicted in the figure, four disks' worth of data are stored, and the extra disks are used to store redundant information for failure recovery.



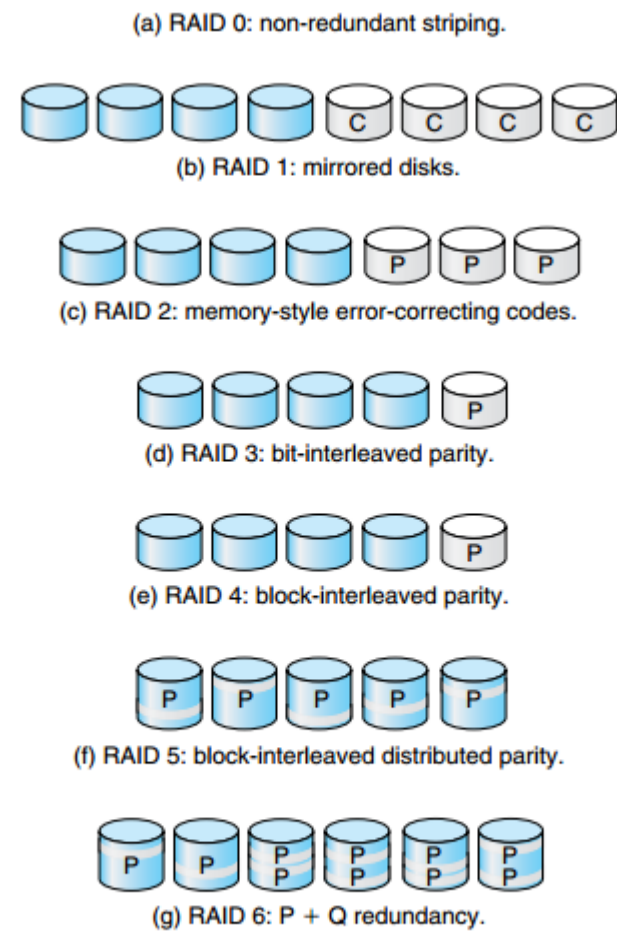


Figure 10.11 RAID levels.

- **RAID level 0.** RAID level 0 refers to disk arrays with **striping** at the level of blocks but **without any redundancy** (such as mirroring or parity bits), as shown in Figure 10.11(a).
- **RAID level 1.** RAID level 1 refers to **disk mirroring**. Figure 10.11(b) shows a mirrored organization.
- **RAID level 2.** RAID level 2 is also known as memory-style error-correcting-code (ECC) organization. Memory systems have long detected certain errors by using parity bits. Each byte in a memory system may have a parity bit associated with it that records whether the number of bits in the byte set to 1 is even (parity = 0) or odd (parity = 1). If one of the bits in the byte is damaged (either a 1 becomes a 0, or a 0 becomes a 1), the parity of the byte changes and thus does not match the stored parity. Similarly, if the stored parity bit is damaged, it does not match the computed parity. Thus, to reconstruct the damaged data. Note that RAID level 2 requires only three disks' overhead for four disks of data, unlike RAID level 1, which requires four disks' overhead.
- **RAID level 3.** RAID level 3, or bit-interleaved parity organization, improves on level 2 by taking into account the fact that, unlike memory systems, disk controllers can detect whether a sector has been read correctly, so a single parity bit can be used for error correction as well as for detection. The idea
- RAID level 3 has two advantages over level 1. First, the storage overhead is reduced because only one parity disk is needed for several regular disks, whereas one mirror disk is needed for every disk in level 1. Second, since reads and writes of a byte are spread out over multiple disks with N-way striping of data, the transfer rate for reading or writing a single block is N times as fast as with RAID level 1. On the negative side, RAID level 3 supports fewer I/Os per second, since every disk has to participate

every I/O request. A further performance problem with RAID 3—and with all parity-based RAID levels—is the expense of computing and writing the parity.

This overhead results in significantly slower writes than with non-parity RAID arrays. To moderate this performance penalty, many RAID storage arrays include a hardware controller with dedicated parity hardware. This controller offloads the parity computation from the CPU to the array. The array has an NVRAM cache as well, to store the blocks while the parity is computed and to buffer the writes from the controller to the spindles. This combination can make parity RAID almost as fast as non-parity. In fact, a caching array doing parity RAID can outperform a non-caching non-parity RAID.

- **RAID level 4.** RAID level 4, or block-interleaved parity organization, uses

higher overall I/O rate. The transfer rates for large reads are high, since all the disks can be read in parallel. Large writes also have high transfer rates, since the data and parity can be written in parallel.

Small independent writes cannot be performed in parallel. An operating-system write of data smaller than a block requires that the block be read, modified with the new data, and written back. The parity block has to be updated as well. This is known as the **read-modify-write cycle**. Thus, a single write requires four disk accesses: two to read the two old blocks and two to write the two new blocks.

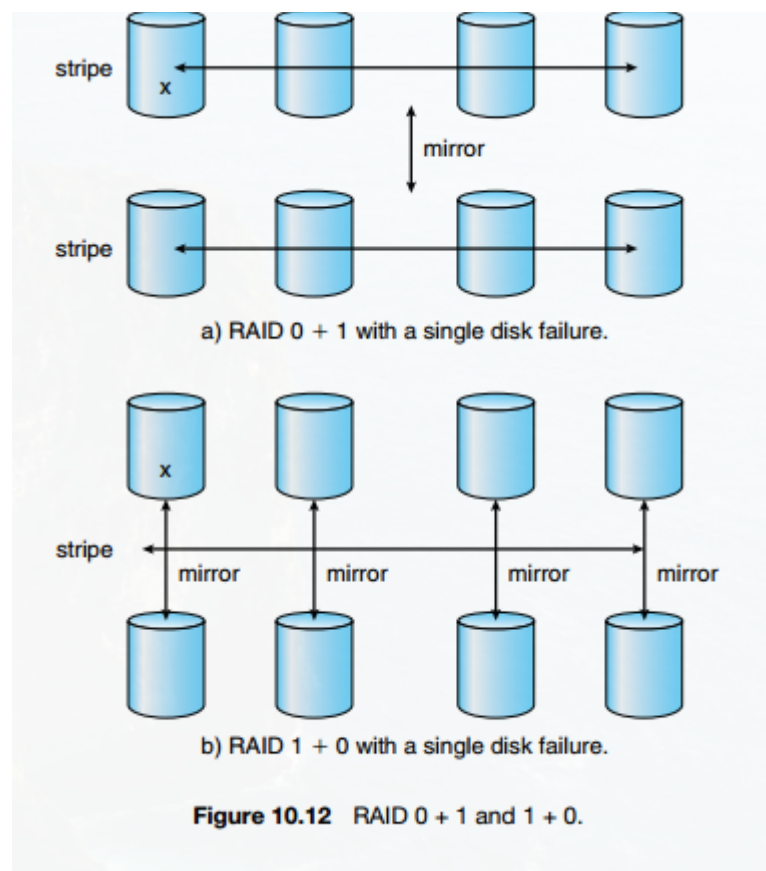
WAFL (which we cover in Chapter 12) uses RAID level 4 because this RAID level allows disks to be added to a RAID set seamlessly. If the added disks are initialized with blocks containing only zeros, then the parity value does not change, and the RAID set is still correct.

- **RAID level 5.** RAID level 5, or block-interleaved distributed parity, differs from level 4 in that it spreads data and parity among all $N + 1$ disks, rather than storing data in N disks and parity in one disk. For each block, one of the disks stores the parity and the others store data. For example, with an array of five disks, the parity for the n th block is stored in disk $(n \bmod 5) + 1$. The n th blocks of the other four disks store actual data for that block. This setup is shown in Figure 10.11(f), where the P s are distributed across all the disks. A parity block cannot store parity for blocks in the same disk, because a disk failure would result in loss of data as well as of parity, and hence the loss would not be recoverable. By spreading the parity across all the disks in the set, RAID 5 avoids potential overuse of a single parity disk, which can occur with RAID 4. RAID 5 is the most common parity RAID system.

- **RAID level 6.** RAID level 6, also called the **P + Q redundancy scheme**, is much like RAID level 5 but stores extra redundant information to guard against multiple disk failures. Instead of parity, error-correcting codes such as the **Reed-Solomon codes** are used. In the scheme shown in Figure

- **RAID levels 0 + 1 and 1 + 0.** RAID level 0 + 1 refers to a combination of RAID levels 0 and 1. RAID 0 provides the performance, while RAID 1 provides the reliability. Generally, this level provides better performance than RAID 5. It is common in environments where both performance and reliability are important. Unfortunately, like RAID 1, it doubles the number of disks needed for storage, so it is also relatively expensive. In RAID 0 + 1, a set of disks are striped, and then the stripe is mirrored to another, equivalent stripe.

Another RAID option that is becoming available commercially is RAID level 1 + 0, in which disks are mirrored in pairs and then the resulting mirrored pairs are striped. This scheme has some theoretical advantages over RAID 0 + 1. For example, if a single disk fails in RAID 0 + 1, an entire stripe is inaccessible, leaving only the other stripe. With a failure in RAID 1 + 0, a single disk is unavailable, but the disk that mirrors it is still available, as are all the rest of the disks (Figure 10.12).



- Implementation of RAID

The implementation of RAID is another area of variation. Consider the following layers at which RAID can be implemented.

- Volume-management software can implement RAID within the kernel or at the system software layer. In this case, the storage hardware can provide minimal features and still be part of a full RAID solution. Parity RAID is fairly slow when implemented in software, so typically RAID 0, 1, or 0 + 1 is used.
- RAID can be implemented in the host bus-adaptor (HBA) hardware. Only the disks directly connected to the HBA can be part of a given RAID set. This solution is low in cost but not very flexible.
- RAID can be implemented in the hardware of the storage array. The storage array can create RAID sets of various levels and can even slice these sets into smaller volumes, which are then presented to the operating system. The operating system need only implement the file system on each of the volumes. Arrays can have multiple connections available or can be part of a SAN, allowing multiple hosts to take advantage of the array's features.
- RAID can be implemented in the SAN interconnect layer by disk virtualization devices. In this case, a device sits between the hosts and the storage. It accepts commands from the servers and manages access to the storage. It could provide mirroring, for example, by writing each block to two separate storage devices.

Other features, such as snapshots and replication, can be implemented at each of these levels as well. A **snapshot** is a view of the file system before the last update took place. (Snapshots are covered more fully in Chapter 12.) **Replication** involves the automatic duplication of writes between separate sites for redundancy and disaster recovery. Replication can be synchronous or asynchronous. In synchronous replication, each block must be written locally and remotely before the write is considered complete, whereas in asynchronous replication, the writes are grouped together and written periodically. Asynchronous replication can result in data loss if the primary site fails, but it is faster and has no distance limitations.

The implementation of these features differs depending on the layer at which RAID is implemented. For example, if RAID is implemented in software, then each host may need to carry out and manage its own replication. If replication is implemented in the storage array or in the SAN interconnect, however, then whatever the host operating system or its features, the host's data can be replicated.

One other aspect of most RAID implementations is a hot spare disk or disks. A hot spare is not used for data but is configured to be used as a replacement in case of disk failure. For instance, a hot spare can be used to rebuild a mirrored

Problems with RAID

integrity of data. ZFS maintains internal checksums of all blocks, including data and metadata. These checksums are not kept with the block that is being checksummed. Rather, they are stored with the pointer to that block. (See Figure 10.13.) Consider an inode — a data structure for storing file system metadata — with pointers to its data. Within the inode is the checksum of each block of data. If there is a problem with the data, the checksum will be incorrect, and the file system will know about it. If the data are mirrored, and there is a block with a correct checksum and one with an incorrect checksum, ZFS will automatically update the bad block with the good one. Similarly, the directory entry that points to the inode has a checksum for the inode. Any problem in the inode is detected when the directory is accessed. This checksumming takes places throughout all ZFS structures, providing a much higher level of consistency, error detection, and error correction than is found in RAID disk sets or standard file systems. The extra overhead that is created by the checksum calculation and extra block read-modify-write cycles is not noticeable because the overall performance of ZFS is very fast.

Another issue with most RAID implementations is lack of flexibility. Consider a storage array with twenty disks divided into four sets of five disks. Each set of five disks is a RAID level 5 set. As a result, there are four separate volumes, each holding a file system. But what if one file system is too large to fit on a five-disk RAID level 5 set? And what if another file system needs very little space? If such factors are known ahead of time, then the disks and volumes

10.7.6 Problems with RAID

Unfortunately, RAID does not always assure that data are available for the operating system and its users. A pointer to a file could be wrong, for example, or pointers within the file structure could be wrong. Incomplete writes, if not properly recovered, could result in corrupt data. Some other process could accidentally write over a file system's structures, too. RAID protects against physical media errors, but not other hardware and software errors. As large as is the landscape of software and hardware bugs, that is how numerous are the potential perils for data on a system.

The Solaris ZFS file system takes an innovative approach to solving these problems through the use of checksums—a technique used to verify the

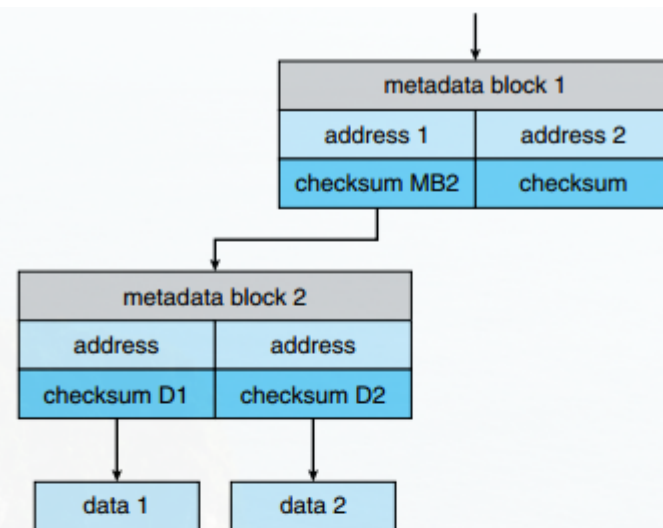


Figure 10.13 ZFS checksums all metadata and data.

can be properly allocated. Very frequently, however, disk use and requirements change over time.

Even if the storage array allowed the entire set of twenty disks to be created as one large RAID set, other issues could arise. Several volumes of

various sizes could be built on the set. But some volume managers do not allow us to change a volume's size. In that case, we would be left with the same issue described above—mismatched file-system sizes. Some volume managers allow size changes, but some file systems do not allow for file-system growth or shrinkage. The volumes could change sizes, but the file systems would need to be recreated to take advantage of those changes.

ZFS combines file-system management and volume management into a unit providing greater functionality than the traditional separation of those functions allows. Disks, or partitions of disks, are gathered together via RAID sets into **pools** of storage. A pool can hold one or more ZFS file systems. The entire pool's free space is available to all file systems within that pool. ZFS uses the memory model of `malloc()` and `free()` to allocate and release storage for each file system as blocks are used and freed within the file system. As a result, there are no artificial limits on storage use and no need to relocate file systems between volumes or resize volumes. ZFS provides quotas to limit the size of a file system and reservations to assure that a file system can grow by a specified amount, but those variables can be changed by the file-system owner at any time. Figure 10.14(a) depicts traditional volumes and file systems, and Figure 10.14(b) shows the ZFS model.

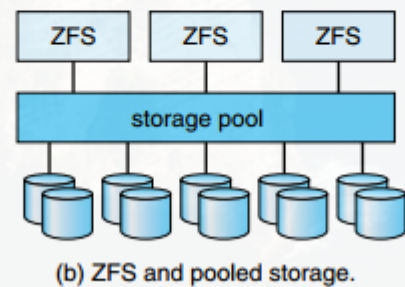
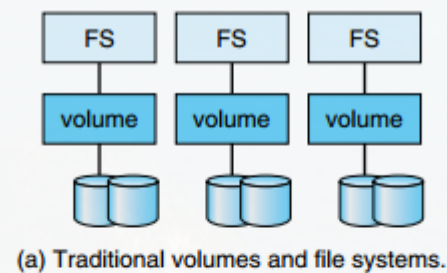


Figure 10.14 (a) Traditional volumes and file systems. (b) A ZFS pool and file systems.

Stable Storage Implementation

A disk write results in one of three outcomes:

1. **Successful completion.** The data were written correctly on disk.
2. **Partial failure.** A failure occurred in the midst of transfer, so only some of the sectors were written with the new data, and the sector being written during the failure may have been corrupted.
3. **Total failure.** The failure occurred before the disk write started, so the previous data values on the disk remain intact.

Whenever a failure occurs during writing of a block, the system needs to detect it and invoke a recovery procedure to restore the block to a consistent state. To do that, the system must maintain two physical blocks for each logical block. An output operation is executed as follows:

1. Write the information onto the first physical block.

2. When the first write completes successfully, write the same information onto the second physical block.
3. Declare the operation complete only after the second write completes successfully.

During recovery from a failure, each pair of physical blocks is examined. If both are the same and no detectable error exists, then no further action is necessary. If one block contains a detectable error then we replace its contents with the value of the other block. If neither block contains a detectable error, but the blocks differ in content, then we replace the content of the first block with that of the second. This recovery procedure ensures that a write to stable storage either succeeds completely or results in no change.

We can extend this procedure easily to allow the use of an arbitrarily large number of copies of each block of stable storage. Although having a large number of copies further reduces the probability of a failure, it is usually reasonable to simulate stable storage with only two copies. The data in stable storage are guaranteed to be safe unless a failure destroys all the copies.

Because waiting for disk writes to complete (synchronous I/O) is time consuming, many storage arrays add NVRAM as a cache. Since the memory is nonvolatile (it usually has battery power to back up the unit's power), it can be trusted to store the data en route to the disks. It is thus considered part of the stable storage. Writes to it are much faster than to disk, so performance is greatly improved.

Highlight	26/08/20 11:30:21
dlostkid	Options
difference between SYNC and ASYNC in FSTAB	

Summary